
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

OPERATION IRON COURIER - Instructor Solution Key

ARM Cortex-M33 • GDB • Ghidra

September 21, 2026

For educational and defensive security research only

The task and criterion headings in this key are word-for-word identical to `ACT-VI-R.md`, so a student can match each criterion one to one.

Artifact Identity

The instructor-issued artifact hashes are:

```
ACT-VI.bin      de54394f53391a8cfc76913e7fd2db5e69ea5dc28bf5ed2e844d39f0855806f0
ACT-VI.uf2      20bc7b7fba79f9c7e507d84ff235efe91ad3b19740b5e7e47c8f83e1125699aa
ACT-VI_fixed.bin 7da0f0c66d25711f9ab793d37da417fd6b0ccd0e35e4ef54ec6a22648d9fd8a5
ACT-VI_fixed.uf2 0bc7b8fa0830828fe1a633283d6efe8dff09c40a5004ba76844f434eef0e166e
```

Machine check: `python scripts/verify_ctf.py` returns 10/10 checks passed against the shipped and corrected images. It asserts the four byte pairs, that only those four offsets differ, and the `ACT-VI.bin` and `ACT-VI_fixed.bin` SHA-256 values. Both `.bin` images are 51,420 bytes and both `.uf2` images are 103,424 bytes.

The four sabotage sites (summary):

Defect	Function	File offset	VA	Compromised	Correct
1 Covert channel	<code>implant_tick</code> (inlined <code>implant_send_channel</code>)	<code>0xA369</code>	<code>0x1000A369</code>	<code>0xD1</code>	<code>0xD0</code>
2 Harvest	<code>implant_tick</code> (inlined <code>implant_stage_write</code>)	<code>0xA329</code>	<code>0x1000A329</code>	<code>0xD1</code>	<code>0xD0</code>
3 Staging marker	<code>implant_init</code> (inlined <code>implant_infect</code>)	<code>0xA5CF</code>	<code>0x1000A5CF</code>	<code>0xB9</code>	<code>0xB1</code>
4 UNLOCK authorization	<code>control_handle_frame</code>	<code>0x757D</code>	<code>0x1000757D</code>	<code>0xB9</code>	<code>0xB1</code>

Task 1: Setup and Initial Analysis (10 points)

Solution

Ghidra Setup. Import `ACT-VI.bin` as Raw Binary, language ARM Cortex 32 little endian default, base address `0x10000000`, then run auto-analysis. The Ghidra project name is `IronCourier_Investigation`. Because every defect is a same-size in-place byte patch, the file offset and the VA differ by exactly `0x10000000` (`VA = offset + 0x10000000`).

Vector Table Decoding. First 32 bytes of `ACT-VI.bin`:

```
00 20 08 20 5D 01 00 10 1B 01 00 10 1D 01 00 10
11 01 00 10 11 01 00 10 11 01 00 10 11 01 00 10
```

Evidence	Answer
Vector table base	<code>0x10000000</code>

Evidence	Answer
Initial SP	0x20082000
Reset handler (as stored)	0x1000015D
Reset instruction address	0x1000015C

The stored reset handler address has bit 0 set, selecting Thumb mode. Clearing bit 0 gives the real entry 0x1000015C.

Entry and Monitor Loop. From ACT-VI-main-disasm.txt :

```

10000234 <main>:
10000234:    b508        push    {r3, lr}
10000236:    f003 fa93    bl      10003760 <stdio_init_all>
1000023a:    4807        ldr     r0, [pc, #28] @ (10000258 <main+0x24>)
1000023c:    f003 fada    bl      100037f4 <__wrap_puts>
10000240:    f006 f8f6    bl      10006430 <monitor_init>
10000244:    b110        cbz     r0, 1000024c <main+0x18>
10000246:    f006 f9bb    bl      100065c0 <monitor_step>
1000024a:    e7fc        b.n     10000246 <main+0x12>

```

Element	Address
main	0x10000234
monitor_init	0x10006430
monitor_step	0x100065C0

Module Map. Anchors for the stripped image:

Module	Anchor function	Address
Entry	main	0x10000234
Monitor / drop-box state machine	monitor_init	0x10006430
Monitor / drop-box state machine	monitor_step	0x100065C0
Control (sealed unlock path)	control_handle_frame	0x10007510
Control (applied command)	control_command	0x10007494
Control (applied package)	control_package	0x100074A0
Locker (actuator)	locker_init	0x100074AC
Locker (actuator)	locker_apply_command	0x100074C4
Locker (actuator)	locker_tick	0x100074F8
Locker (actuator)	locker_fail_safe	0x10007538
Drop-box authorization	dropbox_auth_init	0x10007554
Drop-box authorization	dropbox_auth_set_key	0x10007568

Module	Anchor function	Address
Drop-box authorization	dropbox_auth_apply	0x100075B0
Implant	implant_tick	0x1000A2E4
Implant	implant_handle_command	0x1000A3F0
Implant	implant_init	0x1000A594
Crypto	envelope_open_hex	0x10007858
Radio	radio_send_frame	0x1000A5BC

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Ghidra project created with the correct name and settings	2	Project <code>IronCourier_Investigation</code> , raw binary import
[DOCUMENT] Processor configured as ARM Cortex 32 little endian default	2	Screenshot shows the correct processor
[DOCUMENT] Base address set to 0x10000000	2	Base <code>0x10000000</code>
[DOCUMENT] Vector table, initial stack pointer, and reset handler identified	2	Base <code>0x10000000</code> , initial SP <code>0x20082000</code> , reset handler <code>0x1000015D</code>
[DOCUMENT] main and the drop-box monitor state machine (monitor_step) addresses identified	1	<code>main</code> <code>0x10000234</code> , <code>monitor_step</code> <code>0x100065C0</code>
[DOCUMENT] Module map identifies the locker, control, dropbox_auth, implant, and monitor anchors	1	At least one correct anchor per module

Instructor Notes & Assembly

- Confirm the Ghidra import used `Raw Binary`, `ARM Cortex 32 little endian default`, base `0x10000000`, and that auto-analysis completed before any address was read. In the language dialog the student must search `Cortex` and pick the ARM Cortex 32 little endian default entry.
- Accept either the Import Results Summary or the Program Information window as proof of the name, language, and base address.
- The stored reset handler `0x1000015D` is odd because bit 0 selects Thumb; clearing it gives `0x1000015C`.
- Always say `reset handler`, never `reset pointer`.
- The vector table is identical in the compromised and corrected images because no defect touches it.
- The module map is graded on coverage, not on exhaustive function recovery: one correctly named anchor per module is sufficient. `implant_infect` and `implant_send_channel` are inlined and have no standalone symbol.

Task 2: Bug #1 The Covert Channel (20 points)

Solution

Locate the branch. `implant_tick` starts at `0x1000A2E4` and the channel encoder gate is at file offset `0xA369` (VA `0x1000A369`). The corrected image is:

```

1000a22e:    4b21        ldr     r3, [pc, #132] @ (1000a2b4 <implant_tick+0x100>)
1000a230:    3001        adds    r0, #1
1000a232:    781b        ldrb    r3, [r3, #0]
1000a234:    6030        str     r0, [r6, #0]
1000a236:    2b00        cmp     r3, #0
1000a238:    d0d0        beq.n   1000a1dc <implant_tick+0x28>

```

Instruction decode. `ldr r3, [pc, #132]` loads the channel gate at `0x20013D26` (literal at `0x1000A2D4`), and `ldrb r3, [r3, #0]` reads it. The branch at `0x1000A368` decides whether the encoder may run. The correct code does nothing when the channel gate is clear, so the branch at `0x1000A368` must be `beq (0xD0)` to the `0x1000A30C` return. When the gate is set, the `implant_send_channel` path builds a frame of the ICV1 magic, a one-byte staged count, and the staged records, calls `implant_emit_timing` to spend 250 microseconds per byte, and calls `radio_send_frame` at `0x1000A290`. The condition byte is the high byte at `0x1000A369`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000A369</code>	<code>0xA369</code>	<code>0xD1</code>	<code>bne.n 0x1000A30C</code>	<code>0xD0</code>	<code>beq.n 0x1000A30C</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0xA369</code>	<code>0x1000A369</code>	<code>D0 D1</code>	<code>D0 D0</code>

Why the channel no longer sends. The covert frame is built from the 4-byte ICV1 magic (`0x31564349` little-endian), a one-byte staged-record count, and the eight 7-byte staged records, up to 61 bytes total.

`implant_emit_timing` spends `DROPBOX_IMPLANT_CHANNEL_BIT_US (250)` microseconds for every frame byte, so the payload is the content and the timing is the covert signal, and the radio traffic still looks like ordinary LoRa telemetry. Under the compromised `bne`, the gate is inverted: the fall-through encode path is taken when the gate is clear, so the node emits the preamble and the staged payload on every interval. After the patch, `beq` returns while the gate is clear, so the encoder is skipped and no ICV1 frame is built or timed. Because the encoder reads the raw payload and emits on the same radio as the sealed unlock path, no cryptographic control on the envelope can see or stop it; the only fix is the gate itself.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the covert channel encoder branch at <code>0x1000A369</code>	5	Address and function (<code>implant_tick</code> , inlined <code>implant_send_channel</code>) identified
[DOCUMENT] Documented the ICV1 channel and the timing side channel	5	4-byte ICV1 magic, 250 us per byte, eight-slot ring payload

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT & PATCH] Patched 0xD1 to 0xD0 so the channel does not send	7	Byte 0xD1 changed to 0xD0
[DOCUMENT] Explained why the covert channel needs no cipher and hides in the preamble and timing	3	Raw path under the sealed envelope and timing carries the data

Instructor Notes & Assembly

- The condition byte is the high byte at 0xA369 ; the correct halfword is d0d0 for beq.n and the compromised halfword is d0d1 , so the on-disk bytes are D0 D0 for the fix and D0 D1 for the compromise.
- beq branches when the comparison result is equal (the gate is zero); bne branches when it is not equal. The register holds the channel gate, so the semantics are "do not encode while the gate is clear".
- The channel magic is DROPBOX_IMPLANT_CHANNEL_MAGIC (ICV1), DROPBOX_IMPLANT_CHANNEL_MAGIC_LEN is 4 , the record length is 7 , the staging slots are 8 , and the frame maximum is 61 bytes.
- The timing unit is DROPBOX_IMPLANT_CHANNEL_BIT_US (250).
- Full credit requires both the byte change and a correct statement of the confidentiality lesson: the channel is not a cipher break, it is a side channel beneath the protocol.

Task 3: Bug #2 The Harvest (20 points)

Solution

Locate the branch. implant_tick starts at 0x1000A2E4 and the harvest gate is at file offset 0xA329 (VA 0x1000A329). The corrected image is:

```

1000a1e2:    4930        ldr     r1, [pc, #192] @ (1000a2a4 <implant_tick+0xf0>)
1000a1e4:    7809        ldrb    r1, [r1, #0]
1000a1e6:    2900        cmp     r1, #0
1000a1e8:    d045        beq.n   1000a276 <implant_tick+0xc2>
1000a1ea:    f8d2 2df0   ldr.w   r2, [r2, #3568] @ 0xdf0
1000a1ee:    0792        lsls    r2, r2, #30
1000a1f0:    d141        bne.n   1000a276 <implant_tick+0xc2>
1000a1f2:    4a2d        ldr     r2, [pc, #180] @ (1000a2a8 <implant_tick+0xf4>)
1000a1f4:    7812        ldrb    r2, [r2, #0]
1000a1f6:    2a00        cmp     r2, #0
1000a1f8:    d03d        beq.n   1000a276 <implant_tick+0xc2>

```

Instruction decode. The tick counter at 0x20013748 is advanced. The harvest flag at 0x20013D27 is tested at 0x1000A208 , the CoreDebug DHCSR at 0xE000EDF0 is tested at 0x1000A210 , and ldr r2, [pc, #180] loads the harvest gate at 0x20013D28 (literal at 0x1000A2C8). The branch at 0x1000A328 decides whether a record may be staged. The correct code stages nothing when the harvest gate is clear, so the branch at 0x1000A328 must be beq (0xD0) to the 0x1000A3A6 path. When the gate is set, the inlined implant_stage_write reads a little-endian 16-bit package id from 0x1000 + ticks and an 8-bit attempt count, and implant_stage_commit writes a 7-byte record (package id LE16, attempts u8, tick LE32) into the eight-slot ring at 0x2001370C , wrapping once the ring is full. The condition byte is the high byte at 0x1000A329 .

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
0x1000A329	0xA329	0xD1	bne.n 0x1000A3A6	0xD0	beq.n 0x1000A3A6

Patch.

File Offset	VA	Original Bytes	Patched Bytes
0xA329	0x1000A329	3D D1	3D D0

Why the node no longer harvests. Under the compromised `bne`, the gate is inverted: the fall-through staging path is taken when the gate is clear, so each interval the node appends a synthetic record with a package id of `0x1000 + ticks`, a zero attempt count, and the current tick. After the patch, `beq` returns while the gate is clear, so the ring never grows. Stopping the harvest is a separate control from cutting the channel: the ring is the data store and the channel is the transport, so closing only one leaves either a store with no sender or a sender with stale data. The reserve sector still has to be cleared and the code path removed for a durable fix.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the harvest gate branch at 0x1000A329	5	Address and function (<code>implant_tick</code> , inlined <code>implant_stage_write</code>) identified
[DOCUMENT] Documented the synthetic harvest and the eight-slot staging ring	5	Package LE16, attempts u8, tick LE32, 8 slots
[DOCUMENT & PATCH] Patched 0xD1 to 0xD0 so no synthetic record is staged	7	Byte <code>0xD1</code> changed to <code>0xD0</code>
[DOCUMENT] Explained why a staging ring is a separate control from the channel	3	The ring stores, the channel transports

Instructor Notes & Assembly

- The condition byte is the high byte at `0xA329` ; the correct halfword is `d03d` for `beq.n` and the compromised halfword is `d13d` , so the on-disk bytes are `3D D0` for the fix and `3D D1` for the compromise.
- The stage count is at `0x20013744` , the tick counter is at `0x20013748` , and the eight-slot staging ring is at `0x2001370C` (`DROPBOX_IMPLANT_STAGE_SLOTS * DROPBOX_IMPLANT_RECORD_LEN = 56` bytes).
- The harvest interval is `DROPBOX_IMPLANT_TICK_INTERVAL ( 4 )` , implemented by the low two bits of the tick counter and checked before the harvest gate.
- The `implant_stage_write` path is inlined into `implant_tick` ; there is no standalone symbol in the stripped image.
- Full credit requires both the byte change and a correct statement of the storage-versus-transport lesson.

Task 4: Bug #3 The Staging Marker (20 points)**Solution**

Locate the branch. The `implant_infect` path is inlined into `implant_init` (starts at `0x1000A594`). The marker gate is at file offset `0xA5CF` (VA `0x1000A5CF`). The corrected image is:

```

1000a48e:    7803        ldrb    r3, [r0, #0]
1000a490:    7809        ldrb    r1, [r1, #0]
1000a492:    f1a3 03c7   sub.w   r3, r3, #199 @ 0xc7
1000a496:    fab3 f383   clz     r3, r3
1000a49a:    095b        lsrs    r3, r3, #5
1000a49c:    7013        strb    r3, [r2, #0]
1000a49e:    b1d9        cbz     r1, 1000a4d8 <implant_init+0x74>
1000a4a0:    7803        ldrb    r3, [r0, #0]
1000a4a2:    2bc7        cmp     r3, #199 @ 0xc7
1000a4a4:    d018        beq.n   1000a4d8 <implant_init+0x74>

```

Instruction decode. `ldr r0` loads the reserved sector at `0x103FF000` (literal at `0x1000A51C`), and `ldr r1` loads the marker gate at `0x20013D29` (literal at `0x1000A508`). The `sub.w / clz / lsrs` sequence at `0x1000A4B2` computes `implant_infected()` and stores it in the armed flag at `0x20013D24`. The branch at `0x1000A5CE` decides whether the marker may be written. The correct code writes no marker when the gate is clear, so the branch at `0x1000A5CE` must be `cbz (0xB1)` to the `0x1000A608` return. When the gate is set, a second check at `0x1000A4C4` guards the write, and the Pico SDK flash sequence runs: `strb.w r3, [sp]` stages `0xC7` (`movs r3, #199` at `0x1000A4D8`), then `flash_range_erase` at `0x1000A4E4` and `flash_range_program` at `0x1000A4F0` program the sector. The condition byte is the high byte at `0x1000A5CF`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
0x1000A5CF	0xA5CF	0xB9	cbnz r1, 0x1000A608	0xB1	cbz r1, 0x1000A608

Patch.

File Offset	VA	Original Bytes	Patched Bytes
0xA5CF	0x1000A5CF	D9 B9	D9 B1

The anti-debug obstacle. The implant reads CoreDebug DHCSR at `0xE000EDF0` and returns early while a probe is attached, which suppresses the harvest and the channel:

```

1000a1c8:    f04f 22e0   mov.w   r2, #3758153728 @ 0xe000e000
1000a1cc:    b5f0        push   {r4, r5, r6, r7, lr}
1000a1ce:    f8d2 4df0   ldr.w   r4, [r2, #3568] @ 0xdf0
1000a1d2:    b091        sub    sp, #68 @ 0x44
1000a2E4:    431c        orrs   r4, r3
1000a1d6:    f014 0403   ands.w  r4, r4, #3
1000a1da:    d002        beq.n  1000a1e2 <implant_tick+0x2e>
1000a1dc:    b011        add    sp, #68 @ 0x44
1000a1de:    bdf0        pop    {r4, r5, r6, r7, pc}

```

```

1000a2f6:    f04f 23e0   mov.w   r3, #3758153728 @ 0xe000e000
1000a2fa:    f8d3 3df0   ldr.w   r3, [r3, #3568] @ 0xdf0
1000a2fe:    079b        lsls   r3, r3, #30
1000a300:    d1eb        bne.n  1000a2da <implant_handle_command+0x1a>

```

The mask and shift keep bit 1 (`C_HALT`) and bit 0 (`C_DEBUGEN`) and discard the rest; a non-zero result means a probe is attached and the path returns early. The same register is read again at `0x1000A376` to stamp the frame body. The guard is identical in both images, so it is an analysis obstacle, not one of the four graded defects.

Defeating the anti-debug. Clear the debug bits in the register as seen by the target, or patch the read in a scratch copy. The register is only a view of debug state, so clearing it makes the attach test see no probe. Show the command sequence, not a fabricated transcript; record what the target actually does:

```

arm-none-eabi-gdb ACT-VI.elf
(gdb) target extended-remote /dev/cu.usbmodemXXXX
(gdb) monitor reset halt
(gdb) break implant_init
(gdb) continue
(gdb) set {unsigned int}0xE000EDF0 = 0
(gdb) break *0x1000A4F4
(gdb) continue
(gdb) x/4xb 0x103FF000

```

To observe the boot write on the compromised image, break after the flash program at `0x1000A4F4` (`msr PRIMASK, r4`) in `implant_init`, then read the reserved sector at `0x103FF000` and confirm the first byte is `C7`. To observe the channel and the harvest, clear the debug bits (or patch the `ldr.w` at `0x1000A1EE` in a scratch copy to load a zero constant) and let `implant_tick` run. The scratch copy is for observation only; the shipped artifact is patched at the defect.

Why no marker is written. Under the compromised `cbnz`, the marker gate is inverted: the write path is taken when the gate is clear, so the first boot writes `0xC7` to `0x103FF000`. After the patch, `cbz` returns while the gate is clear, so the flash erase and program at `0x1000A4E4` and `0x1000A4F0` are never reached and the sector stays blank. The marker is the durable state that re-arms the payload handler and the channel on every later boot, and the reserved sector sits outside the program region a firmware reflash writes, which is why the marker survives a reflash and why the gate must be fixed in code, not only erased on the bench.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the staging marker branch at <code>0x1000A5CF</code>	5	Address and inlined <code>implant_init</code> path identified
[DOCUMENT] Documented the CoreDebug DHCSR anti-debug and how it is defeated under GDB	5	<code>0xE000EDF0</code> , <code>C_DEBUGEN</code> and <code>C_HALT</code> , and a real defeat method
[DOCUMENT & PATCH] Patched <code>0xB9</code> to <code>0xB1</code> so no marker is written to <code>0x103FF000</code>	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained the reserved sector <code>0x103FF000</code> and the write-once marker byte <code>0xC7</code>	3	Marker, reserved sector, write-once first run

Instructor Notes & Assembly

- The infect path is inlined into `implant_init`; there is no standalone `implant_infect` symbol in the stripped image.
- The condition byte is the high byte at `0xA5CF`; the correct halfword is `b1d9` for `cbz` and the compromised halfword is `b9d9`, so the on-disk bytes are `D9 B1` for the fix and `D9 B9` for the compromise.
- The marker byte is `DROPBOX_IMPLANT_MARKER_BYTE` (`0xC7`), the reserved sector is `DROPBOX_IMPLANT_RESERVE_ADDR` (`0x103FF000`), and the marker gate is at `0x20013D29`.
- The DHCSR address is `DROPBOX_IMPLANT_DHCSR_ADDR` (`0xE000EDF0`); bit 0 is `C_DEBUGEN` and bit 1 is `C_HALT`. The anti-debug is identical in both images, so it is an analysis obstacle, not one of the four graded defects.
- Grade the GDB point on a real command sequence and the correct observed code path, not on a memorized register dump. Accept either clearing the bits with GDB or patching the read in a scratch copy.
- A common failure is patching the shipped artifact at `0xA5CF` before observing the marker. The order matters: defeat the anti-debug, observe, then patch.

Task 5: Bug #4 The UNLOCK Authorization (20 points)

Solution

Locate the branch. In `control_handle_frame` (starts at `0x10007510`) the authorization branch is at file offset `0x757D` (VA `0x1000757D`). The corrected image is:

```

10007442:    990a      ldr     r1, [sp, #40]    @ 0x28
10007444:    4808      ldr     r0, [pc, #32]    @ (10007468 <control_handle_frame+0x88>)
10007446:    aa06      add     r2, sp, #24
10007448:    f000 f8a2  bl      10007590 <dropbox_auth_apply>
1000744c:    b128      cbz     r0, 1000745a <control_handle_frame+0x7a>
1000744e:    4a07      ldr     r2, [pc, #28]    @ (1000757C <control_handle_frame+0x8c>)
10007450:    4b07      ldr     r3, [pc, #28]    @ (10007470 <control_handle_frame+0x90>)
10007452:    7014      strb    r4, [r2, #0]
10007454:    801d      strh    r5, [r3, #0]
10007456:    b017      add     sp, #92 @ 0x5c
10007458:    bd30      pop     {r4, r5, pc}
1000745a:    2000      movs    r0, #0
1000745c:    b017      add     sp, #92 @ 0x5c
1000745e:    bd30      pop     {r4, r5, pc}

```

Instruction decode. After the sealed frame is opened and the command byte and package are range-checked, `dropbox_auth_apply` verifies the anti-replay sequence window and the authenticated-state tag and returns its authorization verdict in `r0`. The branch at `0x1000757C` decides whether the command may reach the applied command and package. The correct code rejects a failed or replayed authorization, so the branch at `0x1000757C` must be `cbz` (`0xB1`) to the `0x1000758A` reject path, which returns zero. Only a true verdict falls through to `strb r4, [r2, #0]` and `strh r5, [r3, #0]`, which write the accepted command at `0x20013D20` and the package at `0x20013D16`. The condition byte is the high byte at `0x1000757D`.

Address	File offset	Compromised byte	Compromised instruction	Correct byte	Correct instruction
<code>0x1000757D</code>	<code>0x757D</code>	<code>0xB9</code>	<code>cbnz r0, 0x1000758A</code>	<code>0xB1</code>	<code>cbz r0, 0x1000758A</code>

Patch.

File Offset	VA	Original Bytes	Patched Bytes
<code>0x757D</code>	<code>0x1000757D</code>	28 B9	28 B1

Why the command now requires authorization. Under the compromised `cbnz`, the verdict is inverted: a failed or replayed authorization falls through to the stores at `0x1000746E`, while a genuine authorization branches to the reject path and returns zero. After the patch, `cbz` sends a false verdict to the reject path at `0x1000758A`, so an unauthenticated command, a forged command, and a replayed captured command all fail before the command byte and package are applied. A legitimate authorized command still returns true and applies. The rest of the path is correct: the envelope is opened under the field key, the command byte is checked against `DROPBOX_COMMAND_UNLOCK` (`0x01`), `DROPBOX_COMMAND_LOCK` (`0x02`), and `DROPBOX_COMMAND_DENY` (`0x03`), and the package is checked against the band `0` to `16`.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[DOCUMENT] Located the UNLOCK authorization branch at 0x1000757D	5	Address and function (<code>control_handle_frame</code>) identified
[DOCUMENT] Documented the authorization verdict inversion and the branch condition	5	Reject when the verdict is false
[DOCUMENT & PATCH] Patched 0xB9 to 0xB1 so failed and replayed authorizations are rejected	7	Byte <code>0xB9</code> changed to <code>0xB1</code>
[DOCUMENT] Explained why an unauthenticated or replayed UNLOCK envelope must be rejected	3	The applied command must see only an authorized verdict

Instructor Notes & Assembly

- The condition byte is the high byte at `0x757D` ; the correct halfword is `b128` for `cbz` and the compromised halfword is `b928` , so the on-disk bytes are `28 B1` for the fix and `28 B9` for the compromise.
- `dropbox_auth_apply` performs the monotonic anti-replay check and the authenticated-state tag, so this branch is the verdict for both freshness and state integrity.
- Full credit requires the inversion explanation: the compromised build accepts a false verdict and rejects a true one.
- Point out that the rest of the unlock command path is correct. Only the verdict seam was broken.
- This is the defect that is a policy defect rather than an implant behavior, and it is the one a defender would fix first in production.

Task 6: Export and Verify (10 points)

Solution

Export. In Ghidra, File -> Export Program... , choose `Binary Format` , and save as `ACT-VI_fixed.bin` . The shipped image is 51,420 bytes.

Convert.

```
python uf2conv.py ACT-VI_fixed.bin --base 0x10000000 --family 0xe48bff59 --output ACT-VI_fixed.uf2
```

If `uf2conv.py` is not in the working directory, use the copy shipped with the project repository. The UF2 for ACT-VI is 103,424 bytes.

Verify.

```
python scripts/verify_ctf.py
```

Expected result:

```
10/10 checks passed
```

Hardware proof. Flash `ACT-VI_fixed.uf2` in BOOTSEL mode and confirm:

- the reserved sector at `0x103FF000` stays blank after a boot;
- the channel no longer emits the ICV1 preamble or a timing-encoded frame;
- the staging ring no longer grows on the 4-tick interval;
- an unauthenticated command and a replayed captured command are rejected before the command and package are applied;
- a legitimate authorized command still applies, and the courier remote, the courier-arrival button, and the fail-locked policy still behave.

Summary of all patches.

#	Bug	File Offset	Flash Address	Original Byte	Patched Byte
1	The Covert Channel	<code>0xA369</code>	<code>0x1000A369</code>	<code>D1</code>	<code>D0</code>
2	The Harvest	<code>0xA329</code>	<code>0x1000A329</code>	<code>D1</code>	<code>D0</code>
3	The Staging Marker	<code>0xA5CF</code>	<code>0x1000A5CF</code>	<code>B9</code>	<code>B1</code>
4	The UNLOCK Authorization	<code>0x757D</code>	<code>0x1000757D</code>	<code>B9</code>	<code>B1</code>

Reflection mapping. The four defects map to real control-system failures:

Defect	Real-world failure
The Covert Channel	A payload hides data in the preamble and timing of legitimate traffic, so confidentiality fails even when every packet is well formed.
The Harvest	A payload quietly collects records into a staging store, so the device becomes a collector as well as a controller.
The Staging Marker	A payload writes a durable marker to a reserved sector, so the state that re-arms it survives remediation.
The UNLOCK Authorization	An inverted verdict lets an unauthenticated or replayed command change a physical command and package.

Grading Rubric (1-to-1 Mapping)

Criterion	Points	Full Credit (Answer Key)
[PATCH] Exported <code>ACT-VI_fixed.bin</code> from Ghidra	2	Valid patched binary
[PATCH] Converted to <code>ACT-VI_fixed.uf2</code> with the correct base and family	2	<code>--base 0x10000000 --family 0xe48bff59</code>
[DOCUMENT] <code>scripts/verify_ctf.py</code> passes and hardware proves the correct behavior	3	Verifier passes and the hardware proof is shown
[DOCUMENT] Reflection maps each of the four defects to a real-world control-system failure	3	Specific mapping for all four

Instructor Notes & Assembly

- Confirm the exported image differs from `ACT-VI.bin` in exactly the four bytes in the table; `scripts/verify_ctf.py` checks this and the SHA-256 values.
- Confirm the UF2 conversion used base `0x10000000` and family `0xe48bff59`.
- The shipped image is 51,420 bytes; the corrected image must be the same size because every patch is in place.
- Grade the reflection on specificity, not length: each of the four defects should name a concrete control-system consequence.
- Remind students that the anti-debug is not patched out of the shipped artifact; only the four defect bytes change.

How To Breadboard

Device	Pin on device	Pico 2 GPIO	Notes
DHT11 locker climate sensor	DATA	GP4	10 kOhm pull-up to 3.3 V if the module needs it
1602 LCD	SDA	GP2	I2C1, backpack address <code>0x27</code>
1602 LCD	SCL	GP3	I2C1, 100 kHz
1602 LCD	VCC / GND	VBUS 5 V / GND	The backpack needs 5 V, not 3.3 V
RYLR998	RX	GP8 (Pico TX)	UART1, 115200, network ID 18
RYLR998	TX	GP9 (Pico RX)	UART1
IR receiver	OUT	GP5	VS1838B, internal pull-up enabled
Servo	signal	GP14	PWM 50 Hz; 1000 uF bulk cap across servo 5 V and GND
Red LED	anode	GP16	DENIED, 220 to 330 ohm to GND
Yellow LED	anode	GP17	COURIER WAITING, 220 to 330 ohm to GND
Green LED	anode	GP18	UNLOCKED, 220 to 330 ohm to GND
Courier-arrival button	leg 1	GP15	Internal pull-up; leg 2 to GND, never to 3.3 V
Onboard LED	built in	GP25	Heartbeat
Debug Probe	SWCLK / SWDIO / GND	debug header	For GDB only

Use 3.3 V logic on every GPIO. The only 5 V connection is the LCD backpack supply. Keep the 1000 uF capacitor on the servo rail to absorb the SG90 current spike.

Complete Grading Summary

Task	Title	Points
Task 1	Setup and Initial Analysis	10
Task 2	Bug #1 The Covert Channel	20
Task 3	Bug #2 The Harvest	20
Task 4	Bug #3 The Staging Marker	20
Task 5	Bug #4 The UNLOCK Authorization	20
Task 6	Export and Verify	10
TOTAL		100

Instructor Notes

Safety: Use only the supplied Pico 2, Debug Probe, and firmware. Never connect the exercise to an operational logistics network, a pharmaceutical network, a building-management system, a public network, a military system, or a third-party device.

Common Student Mistakes

- Patching the low byte of the branch at `0xA258`, `0xA218`, `0xA4BE`, or `0x746C` instead of the condition byte at `0xA369`, `0xA329`, `0xA5CF`, or `0x757D`.
- Reading the channel gate or the harvest gate backwards and believing the corrected build still sends or still stages.
- Searching for a standalone `implant_infect` or `implant_send_channel` symbol and missing that both are inlined into `implant_init` and `implant_tick`.
- Treating the CoreDebug `DHCSR` anti-debug as a defect and trying to patch it, when it is identical in both images and is an analysis obstacle.
- Patching the shipped artifact before observing the marker write, so the payload is never demonstrated.
- Reversing the authorization explanation: under the compromise the accept path is taken when the verdict is false.
- Confusing `cbz` and `cbnz` on the two clearing gates.
- Forgetting that the fix for the marker is two parts: the patch and the reserved-sector erasure.
- Forgetting the UF2 conversion or using the wrong family flag.
- Fabricating a GDB session instead of showing the command sequence and the real observed code path.

Partial Credit Guidelines

- Award partial credit for a correct address without the correct byte, or a correct byte without the address.
- Award partial credit for documented before/after bytes without the control-flow explanation, or vice versa.
- Award partial credit for a correct GDB command sequence without a clear statement of the observed code path, or the observation without the commands.

- Award partial credit for a correct anti-debug explanation without a working defeat method, or a working method without the explanation.
- Award partial credit for naming the reserved sector and the marker without the persistence lesson, or the lesson without the addresses.
- Award no credit for patches that alter any byte outside the four documented offsets, and no credit for a fabricated GDB session.

Appendix: Expected Binary Diff

These offsets are from the compiled image loaded at `0x10000000`.

```
--- ACT-VI.bin (compromised)
+++ ACT-VI_fixed.bin (corrected)

Offset 0x0000757D: B9 -> B1 (cbnz r0, 0x1000758A -> cbz r0, 0x1000758A)
Offset 0x0000A329: D1 -> D0 (bne.n 0x1000A3A6 -> beq.n 0x1000A3A6)
Offset 0x0000A369: D1 -> D0 (bne.n 0x1000A30C -> beq.n 0x1000A30C)
Offset 0x0000A5CF: B9 -> B1 (cbnz r1, 0x1000A608 -> cbz r1, 0x1000A608)
```

#	Bug	File Offset	Flash Address	Original Bytes	Patched Bytes
1	The Covert Channel	0xA369	0x1000A369	D0 D1	D0 D0
2	The Harvest	0xA329	0x1000A329	3D D1	3D D0
3	The Staging Marker	0xA5CF	0x1000A5CF	D9 B9	D9 B1
4	The UNLOCK Authorization	0x757D	0x1000757D	28 B9	28 B1

Four defects, four changed bytes in four instructions: the covert channel encoder gate, the harvest gate, the staging marker gate, and the authorization verdict. No other byte in either image differs.